

Gamified Learning Platform for Rural Education with AI Visual Conversion



A

Project Report

Submitted in partial fulfillment of the requirement for the award of degree of

Bachelor of Technology

In

Information Technology

Submitted to

RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA,

BHOPAL (M.P.)

Guided By

Prof. Deepak Singh Chauhan

Submitted By

Prashant Jagtap (0827IT231101)

Sarthik Yadav (0827IT231120)

Shreyansh Sharma (0827IT231130)

Somil Jain (0827IT231135)

**DEPARTMENT OF INFORMATION TECHNOLOGY
ACROPOLIS INSTITUTE OF TECHNOLOGY & RESEARCH,**

INDORE (M.P.) 453771

2025-2026

Declaration

We hereby declared that the work, which is being presented in the project entitled **Gamified Learning Platform For Rural Education with Ai Visual Conversion** partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology**, submitted in the department of Information Technology at **Acropolis Institute of Technology & Research, Indore** is an authentic record of my own work carried under the supervision of **Prof. Deepak Singh Chouhan**. I have not submitted the matter embodied in this report for the award of any other degree.

Prashant Jagtap (0827IT231101)
Sarathik Yadav (0827IT231120)
Shreyansh Sharma (0827IT231130)
Somil Jain (0827IT231135)

Prof. Deepak Singh Chouhan
Supervisor

Project Approval Form

I hereby recommend that the project **Gamified Learning Platform For Rural Education with AI Visual Conversion** prepared under my supervision by Prashant Jagtap (0827IT231101), Sarthik Yadav (0827IT231120), Shreyansh Sharma (0827IT231130), Somil Jain (0827IT231135) be accepted in partial fulfillment of the requirement for the degree of Bachelor of Technology in Information Technology.

Prof. Deepak Singh Chouhan

Supervisor

Recommendation concurred in 2025-2026

Prof. Pawan Kumar Gupta

Project Incharge

Prof. Deepak Singh Chouhan

Project Coordinator

Acropolis Institute of Technology & Research

Department of Information Technology



Certificate

The project work entitled **Gamified Learning Platform For Rural Education with AI Visual Conversion** submitted by Prashant Jagtap (0827IT231101), Sarthik Yadav (0827IT231120), Shreyansh Sharma (0827IT231130), Somil Jain (0827IT231135) is approved as partial fulfillment for the award of the degree of Bachelor of Technology in Information Technology by Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.).

Internal Examiner

Name:.....

Date:/.../.....

External Examiner

Name:

Date:/.../.....

Acknowledgement

With boundless love and appreciation, we would like to extend our heartfelt gratitude and appreciation to the people who helped us to bring this work to reality. We would like to have some space of acknowledgement for them.

Foremost, our would like to express our sincere gratitude to our supervisor, **Prof. Deepak Singh Chouhan** whose expertise, consistent guidance, ample time spent and consistent advice that helped us to bring this study into success.

To the project in-charge **Prof. Pawan Kumar Gupta** and project coordinator **Prof. Deepak Singh Chouhan** for their constructive comments, suggestions, and critiquing even in hardship.

To the honorable **Prof. (Dr.) Prashant Lakkadwala**, Head, Department of Information Technology for his favorable responses regarding the study and providing necessary facilities.

To the honorable **Dr. S.C. Sharma**, Director, AITR, Indore for his unending support, advice and effort to make it possible.

Finally, we would like to pay our thanks to faculty members and staff of the Department of Computer Science & Engineering for their timely help and support.

We also like to pay thanks to our **parents** for their eternal love, support and prayers without them it is not possible.

Prashant Jagtap (0827IT231101)

Sarthik Yadav (0827IT231120)

Shreyansh Sharma (0827IT231130)

Somil Jain (0827IT231135)

ABSTRACT

Rural classrooms in India continue to rely on text-heavy instruction, chalkboard dictation, and periodic tests that arrive too late to be useful. Digital platforms have grown rapidly over the past decade, but they have mostly solved the wrong problem — delivering polished video content to urban learners with reliable broadband, while leaving rural teachers' hand-prepared notes sitting in notebooks.

This project presents the design and partial implementation of a Gamified Learning Platform for Rural Education with AI Visual Conversion. The platform accepts what teachers already produce — typed notes, scanned PDFs, photographed blackboard images, and voice recordings — and converts that raw material into structured visual learning artifacts: concept maps, process flowcharts, and summary infographics. The conversion pipeline is built on open-source transformer-based Natural Language Processing (NLP) models, specifically multilingual BERT variants and KeyBERT for keyphrase extraction, combined with lightweight graph layout algorithms to generate SVG-based visual outputs.

The visual artifacts are embedded within a gamified learning loop. Students earn experience points (XP) for completing quizzes generated automatically from the same content. Streak counters, badge awards, and weekly class leaderboards provide motivational scaffolding grounded in Self-Determination Theory. The system runs without continuous internet access through an offline-first architecture using Django service workers and local caching, making it practical on the intermittent 2–5 Mbps connections typical of rural Madhya Pradesh and Rajasthan.

Four role-based dashboards — student, teacher, guardian, and administrator — surface analytics that help teachers identify struggling learners before examination season. The lagging-learner detection module flags students using three configurable heuristics: inactivity over five school days, XP below 30 percent of class median, and quiz accuracy below 40 percent in any subject across the last five attempts.

This report documents the complete system: motivation, literature survey, theoretical foundations, system architecture, module design, UML documentation, algorithms, implementation details, and a pilot evaluation plan. The UML suite includes a Class Diagram, Sequence Diagram, Use Case Diagram, Data Flow Diagram, and State Diagram, all of which are presented as appendices.

Keywords: *gamification, rural education, artificial intelligence, NLP, text-to-visual conversion, offline learning, Django, concept maps, formative assessment, XP, badges.*

TABLE OF CONTENTS

ABSTRACT	6
LIST OF FIGURES	10
LIST OF TABLES	11
CHAPTER 1	12
INTRODUCTION	12
1.1 Background and Motivation	12
1.2 Problem Statement	13
1.3 Objectives of the Project	14
1.4 Scope and Limitations	15
1.5 Report Organisation	16
CHAPTER 2	17
LITERATURE SURVEY	17
2.1 Gamification in Rural Education	17
2.2 AI-Powered Language Learning Systems	18
2.3 Offline-First AR/AI/GBL Systems	19
2.4 Web-Based Gamified Platforms	19
2.5 Comparative Analysis	20
2.6 Research Gaps	21
CHAPTER 3	22
THEORETICAL ANALYSIS	22
3.1 Self-Determination Theory	22
3.2 Mayer's Multimedia Learning Theory	22
3.3 Constructivism	23
3.4 Universal Design for Learning	24
3.5 Block Diagram	25
CHAPTER 4	26
SYSTEM DESIGN	26
4.1 System Architecture Overview	26
4.2 Layered Architecture	26
4.2.1 Presentation Layer	26
4.2.2 Application Layer	27
4.2.3 Data Layer	27
4.2.4 AI Services Layer	28
4.2.5 Offline and Sync Layer	28
4.3 Deployment Scenarios	29
4.3.1 School-Server Deployment	29

4.3.2 Cloud-Backed Deployment	29
4.4 Hardware and Software Requirements	29
CHAPTER 5	31
MODULE DESIGN	31
5.1 Authentication and Role Management	31
5.2 Content Ingestion and Processing	31
5.3 NLP Engine and Visual Generator	32
5.3.1 Keyphrase Extraction	32
5.3.2 Relationship Inference	33
5.3.3 Summarisation	33
5.3.4 Visual Generator	33
5.4 Gamification Engine	34
5.4.1 XP and Levels	34
5.4.2 Badge System	34
5.4.3 Streaks	35
5.4.4 Leaderboards	35
5.5 Quiz and Assessment Module	35
5.6 Analytics and Dashboards	36
5.6.1 Student Dashboard	36
5.6.2 Teacher Dashboard	36
5.6.3 Guardian Dashboard	37
5.6.4 Administrator Dashboard	37
5.7 Offline and Sync Layer	37
ALGORITHMS AND WORKFLOWS	38
6.1 Text-to-Concept-Map Algorithm	38
6.2 XP Calculation Formula	39
6.3 Lagging Learner Detection	40
6.4 Quiz Generation Pipeline	41
UML DOCUMENTATION	43
7.1 Class Diagram	43
7.2 Use Case Diagram	44
7.3 Sequence Diagram	45
7.4 Data Flow Diagram	46
7.5 State Diagram	46
CHAPTER 8	48
IMPLEMENTATION	48
8.1 Technology Stack	48
8.1.1 Backend Framework	48

8.1.2 Asynchronous Task Processing	48
8.1.3 NLP Models	48
8.1.4 Database and Storage	49
8.1.5 Frontend Implementation	49
8.2 Performance Considerations	50
8.3 Security and Privacy	50
CHAPTER 9	52
EVALUATION PLAN	52
9.1 Research Questions	52
9.2 Pilot Design	52
9.3 Quantitative Metrics	53
CHAPTER 10	55
CONCLUSION AND FUTURE WORK	55
10.1 Conclusion	55
10.2 Future Work	56
REFERENCES	58

LIST OF FIGURES

Fig. 1. Class Diagram	43
Fig. 2. Use Case Diagram	44
Fig. 3. Sequence Diagram	45
Fig. 4. Data Flow Diagram	46
Fig. 5. State Diagram (Stage Zero)	47

LIST OF TABLES

TABLE I Comparative Analysis of Existing Systems	20
TABLE II Hardware Requirements	29
TABLE III Software Requirements	30
TABLE IV Default Badge Criteria	35
TABLE V XP AWARD VALUES BY ACTION TYPE	40
TABLE VI Quantitative Evaluation Metrics	53

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Rural education in India sits at the intersection of two stubborn problems: a shortage of qualified teachers and a near-total absence of engaging learning materials designed for the specific circumstances of village schools. Schools in districts such as Sheopur, Sidhi, and Barwani in Madhya Pradesh operate with student-to-teacher ratios exceeding 40:1 in some grades. The teachers who are present are often generalists covering four or five subjects simultaneously, with no time to prepare separate visual aids for each lesson. Instruction defaults to chalkboard dictation and rote memorisation because those are the only tools reliably available.

This is not a new observation. Government reports, academic studies, and journalism have documented it for decades. What has changed is the technical feasibility of addressing it differently. Two developments that were largely confined to research environments five years ago are now available through open-source libraries: transformer-based NLP models that can extract key concepts from arbitrary text, and offline-capable web architectures that can serve rich interactive applications over 2G connections. Combining these with a well-designed gamification layer produces something that has not been built for this context before — a platform that converts what teachers already write into visual learning materials and delivers them in a way that keeps students coming back.

The motivation behind this project came from a specific frustration: most educational technology built for India is built for the wrong India. Khan Academy, BYJU'S, and Coursera have produced impressive content, but it assumes reliable broadband, a personal device, and learners who already have the study habits to seek out supplementary material. A student in a village school shares two tablets with fourteen classmates and loses mobile data signal every other hour. The platform described in this report was designed with that student in mind, not as an afterthought but as the primary design constraint.

Gamification has attracted serious academic attention since Deterding et al. formalised its definition in 2011. The empirical record is cautious: gamification that goes no deeper than surface badges and points can fail or actively reduce intrinsic motivation. What works is gamification tied to content that learners find genuinely useful, delivering feedback that is immediate and informative, and giving learners meaningful choices about how they engage. The visual conversion engine in this platform is the content layer that makes gamification worthwhile. A student earns XP not for clicking through slides but for completing a quiz generated from their own teacher's notes, a distinction that connects the motivational mechanics to actual learning activity.

The broader context includes India's National Education Policy 2020, which explicitly calls for increased use of technology in education, and the PM e-VIDYA initiative which has accelerated distribution of tablets and low-cost connectivity to rural schools. These policy changes improve the hardware situation, but they do not supply the software layer that makes the hardware useful in a classroom where the teacher cannot spend three hours authoring interactive content. This project addresses that gap.

1.2 Problem Statement

Rural classrooms face several interrelated problems that no single existing platform has solved together:

- Text saturation: Nearly all instructional content is delivered as text — dictated notes, printed textbooks, or chalkboard writing. Students in science and mathematics subjects rarely encounter a concept map, a process diagram, or a labeled illustration unless the textbook happens to include one.
- No automatic visual generation: Teachers who want visual aids must create them manually. In a school with no dedicated art materials and a six-period teaching day, this almost never happens.
- Assessment lag: Formal tests occur monthly or at term end. Between those events, neither teacher nor student has reliable information about where understanding has broken down.

- Connectivity constraints: Mobile data speeds in rural Madhya Pradesh average 2–5 Mbps with frequent dropouts. Systems requiring continuous connectivity for basic operation are unusable in these conditions.
- Language barriers: Students and teachers commonly work in Hindi or regional dialects. English-only platforms add a language barrier on top of the content barrier.
- Device scarcity: A school computer lab may have ten to twenty machines shared across several hundred students. Personal smartphone ownership at the secondary level is variable and limited by family income.

Existing platforms have addressed some of these problems individually. The Django-based Gamified Learning Platform developed by Tejaswi et al. (2026) implements XP, badges, and role-based dashboards in an architecture built for low bandwidth — but all content is centrally authored. Teachers must still write game activities from scratch. Farm and Learn by De Silva et al. (2026) demonstrates successful offline-first architecture with AI components for agricultural education — but it does not convert arbitrary textual notes into visual artifacts. Khan Academy and BYJU'S deliver high-quality content — but it is their content, not the local teacher's.

This platform treats the absence of automatic text-to-visual conversion as the primary problem to solve, with gamification and offline operation as the delivery mechanism that makes the solution usable.

1.3 Objectives of the Project

The platform is designed to accomplish six specific objectives:

1. Accept diverse input formats — typed text, PDFs, scanned images, photographs of handwritten notes, and voice recordings — and produce structured visual learning objects automatically, with no manual diagram authoring required from teachers.
2. Integrate a gamification layer — XP, levels, badges, streaks, and class leaderboards — tied to meaningful educational actions rather than trivial page-view events.

3. Generate quiz questions automatically from the same content used to produce visual artifacts, creating a closed loop between content ingestion and formative assessment.
4. Support multilingual and voice-based interaction so that students and teachers working in Hindi or regional languages are not forced to use English.
5. Operates without continuous internet access through an offline-first architecture: generated content, quiz banks, and gamification state are cached locally and synchronised when connectivity permits.
6. Provide role-based dashboards for students, teachers, guardians, and administrators that surface actionable analytics — patterns of topic difficulty, learner inactivity, and streak maintenance — in a form that a non-technical teacher can interpret and act on.

1.4 Scope and Limitations

This project covers the full design, partial implementation, and detailed evaluation plan for the platform described above. The current scope includes:

- Complete system architecture across five layers: presentation, application, data, AI services, and offline sync.
- Detailed module specifications for authentication, content ingestion, NLP engine, visual generator, gamification engine, quiz module, analytics dashboards, and offline sync.
- Core algorithms: text-to-concept-map conversion, XP calculation, lagging-learner detection, and quiz generation pipeline.
- Full UML documentation suite: Class Diagram, Sequence Diagram, Use Case Diagram, Data Flow Diagram, and State Diagram.
- Reference implementation in Python/Django with Celery, Redis, PostgreSQL, and Hugging Face Transformers.
- Detailed pilot evaluation plan targeting three rural schools in Madhya Pradesh.

The primary limitations at this stage of development are:

- OCR accuracy on low-quality photographs of handwritten notes is below 80 percent for many real-world samples, meaning teachers whose notes are photographed rather than typed will receive lower-quality visual artifacts.
- Automatically generated multiple-choice distractors are not yet reliable enough to publish without teacher review. The current approach requires a review step before quizzes are made available to students.
- The platform currently supports Hindi and English. Adding support for Tamil, Telugu, Kannada, Bengali, and Odia requires fine-tuning NLP models on curriculum-aligned text in each language, which is planned as a subsequent development phase.
- Voice response evaluation for quiz answers is a planned extension and is not included in the current implementation.

1.5 Report Organisation

This report is organised into ten chapters. Chapter 2 surveys the literature on gamification in rural education, AI-powered language learning, offline-first educational systems, and web-based gamified platforms, closing with a comparative analysis that identifies the research gaps this project addresses. Chapter 3 covers the theoretical foundations: Self-Determination Theory, Mayer's Multimedia Learning Theory, Constructivism, and Universal Design for Learning. Chapter 4 describes the overall system design and architecture. Chapter 5 specifies each major module in detail. Chapter 6 presents the core algorithms. Chapter 7 documents the complete UML suite. Chapter 8 covers implementation choices. Chapter 9 presents the pilot evaluation plan. Chapter 10 concludes with a summary and future work roadmap.

CHAPTER 2

LITERATURE SURVEY

2.1 Gamification in Rural Education

The theoretical grounding for gamification in education was formalised by Deterding, Dixon, Khaled, and Nacke in 2011, who defined it as the application of game design elements in non-game contexts. Their taxonomy distinguishes between surface patterns such as points and badges, deeper game mechanics such as challenge and progression, and the underlying design principles that determine whether gamification produces genuine engagement or merely mimics the appearance of it.

Hamari, Koivisto, and Sarsa synthesised 24 empirical studies in 2014 and found that the majority reported positive effects on motivation and engagement, though effect sizes varied considerably with context. Studies involving younger learners, short-duration interventions, and subjects with clear correct answers — mathematics, language vocabulary — tended to show stronger results. Science concepts with complex causal structures produced more mixed outcomes.

The necessary corrective to optimistic readings of this literature comes from Dichev and Dicheva (2017), who reviewed gamification in higher education and found that the field suffers from weak experimental designs, short study durations, and a tendency to report positive outcomes selectively. Their central argument is that cosmetic gamification — badges attached to dull content — can reduce autonomy and shift motivation from intrinsic to extrinsic, which eventually degrades performance. This finding directly shaped the design of the gamification layer in this platform: XP is tied to content interaction quality, leaderboards are opt-in, and badge criteria are tied to mastery milestones rather than time-on-platform.

Fransiska (2024) studied gamification in a rural village in South Lampung, Indonesia, where students from low-income families had limited access to educational technology. Even without digital infrastructure, game-based activities — role-play, word games, team challenges — produced significant improvements in English vocabulary and classroom participation over a single semester. The study's most important finding for this project is

its implication that the benefit of gamification is motivational, not technological. Digital implementation can amplify these benefits but is not required for them to function.

In the Indian context, Suchitha (2025) examined AI-powered English language learning tools for rural learners and identified intelligent tutoring systems and adaptive learning platforms as promising mechanisms for partially compensating for the shortage of trained English teachers in rural areas. The study also highlighted the barriers: cost, infrastructure limitations, teacher digital literacy gaps, and data privacy concerns. This work informed the teacher training and adoption strategy described in the evaluation plan in Chapter 9.

2.2 AI-Powered Language Learning Systems

Commercial AI-enabled language learning platforms — Duolingo, Babbel, Grammarly, Hello English — have built sophisticated adaptive difficulty systems, automated grammar correction, and speech recognition-based pronunciation scoring. Empirical studies suggest these tools increase practice time and lower anxiety when used as supplements to formal instruction (Godwin-Jones, 2020; Johnson, 2019). The platform designs contain valuable patterns for gamification mechanics and AI-assisted learning that informed this project's quiz generation and XP systems.

The limitation is deployment context. These platforms were engineered for urban users with personal smartphones, stable connectivity, and relatively high digital literacy. Their content is centralized and global in focus. They do not ingest teacher-prepared notes and return structured visual lessons. They are not designed for a school in Barwani where fifteen students share two tablets on a 2G connection.

Singh (2022) examined blended learning in rural Indian schools and found that combining AI-based practice tools with teacher-led instruction allowed students to work more independently and receive timely feedback. However, this study's implementation depended on computer labs and tablet deployment, and no component of it addressed the transformation of teacher notes into visual content. The language learning literature as a whole demonstrates the promise of AI and gamification while revealing a consistent gap: systems rarely operate fully offline, and none are designed to process arbitrary curricular notes provided by local teachers.

2.3 Offline-First AR/AI/GBL Systems

De Silva, Prabodhani, and Herath (2026) developed Farm and Learn, an offline-first mobile learning system integrating Augmented Reality, AI, and game-based learning to teach agricultural concepts — specifically paddy cultivation — to children in low-connectivity environments in Sri Lanka. The system includes an AR module for visualising 3D farming equipment, an AI module using a lightweight YOLOv11n model for paddy growth-stage identification from camera images, and game-based quizzes and mini-games.

The architecture adopts a modular, offline-first design: all assets and logic are packaged within the application, eliminating any dependence on cloud services for basic function. Evaluations with students, teachers, and agricultural experts reported high usability and strong engagement, with offline accessibility receiving particularly high ratings. The system demonstrates that carefully optimised AI models can achieve real-time performance on mid-range devices without network connectivity.

For this project, Farm and Learn serves as the direct inspiration for the offline-first architecture and the offline synchronisation model: full content download when WiFi is available, complete offline operation otherwise. The key difference is that Farm and Learn focuses on pre-authored content in a single domain, while this platform processes arbitrary textual input across all school subjects and converts it into visual artifacts. The AI component in Farm and Learn targets visual plant recognition (computer vision); the AI component here targets natural language understanding — a fundamentally different technical challenge.

2.4 Web-Based Gamified Platforms

Tejaswi et al. (2026) built a Gamified Learning Platform for Rural Education implemented as a full-stack Django web application. The platform supports XP, levels, badges, and streaks, with four user roles (student, teacher, administrator, guardian), role-based dashboards, and analytics for tracking academic progress and identifying underperforming students. Pilot testing with rural learners showed high streak preservation rates, successful badge acquisition, and accurate early identification of struggling students.

A notable design decision is the constraint-first architecture: server-side rendering via Django templates, minimal JavaScript, and carefully optimised pages to ensure low bandwidth usage on 4G networks. This stands in contrast to many modern web applications that depend on heavy client-side frameworks. The same architectural philosophy underlies this project's presentation layer.

The gap that Tejaswi et al.'s platform does not address is the one this project treats as central: automatic generation of visual content from teacher notes. All learning content in their system is centrally authored through game modules. Teachers must still write activities manually. This project's NLP engine and visual generator are the components that fill this gap.

2.5 Comparative Analysis

TABLE I: COMPARATIVE ANALYSIS OF EXISTING SYSTEMS RELEVANT TO RURAL GAMIFIED AI LEARNING

System	Gamification	AI Components	Offline Support	Text-to-Visual
Fransiska (2024) — Rural English	Points, levels, manual rewards	None	Fully offline, classroom	Not supported
AI English Learning — Suchitha (2025)	Streaks, leaderboards (varies)	Intelligent tutoring, speech recognition	Typically online	Not supported
Farm and Learn — De Silva (2026)	Quiz scores, mini-games	YOLO plant recognition, AR	Strong offline-first	Not supported
Django Gamified Platform — Tejaswi (2026)	XP, levels, badges, streaks	AI chatbot via external API	Low-bandwidth web	Not supported
This Project	XP, levels, badges, streaks, leaderboards	NLP keyphrase, summarisation, quiz generation, visual layout	Offline-first, local sync	Core capability

As Table I shows, none of the reviewed systems simultaneously supports robust gamification, AI-based text understanding, automated visual content generation from user-uploaded notes, and offline operation designed for rural deployment. This is the design space this project occupies.

2.6 Research Gaps

The synthesis of existing work identifies several open questions that motivated the design choices in this project:

- No system converts locally produced teacher or student notes into visual learning artifacts automatically. All existing platforms either deliver pre-authored centrally produced content or require teachers to author interactive content from scratch.
- Gamification and visual learning have been studied independently but rarely integrated into a single deployable system optimised for rural constraints. The combination is what this platform attempts.
- Offline-first architectures for educational AI systems are understudied. Farm and Learn is the closest prior example, but it operates in a single pre-defined domain rather than across arbitrary curricular content.
- Teacher buy-in and adoption workflows are rarely treated as engineering problems. Most platforms assume teachers will use them if the features are there. The teacher-review workflow and the positioning of AI as a time-saving assistant rather than an authority are design responses to this gap.

CHAPTER 3

THEORETICAL ANALYSIS

3.1 Self-Determination Theory

Deci and Ryan's Self-Determination Theory (SDT) identifies three psychological needs that, when met, support intrinsic motivation: competence (feeling effective at what one does), autonomy (feeling that one's actions are self-chosen), and relatedness (feeling connected to others who matter). Gamification research uses SDT as its primary theoretical frame because it explains both why gamification sometimes works and why it sometimes backfires.

Mechanics that support competence and autonomy — progressive challenge, meaningful choice, informative feedback — sustain motivation over time. Mechanics that undermine autonomy — mandatory competition, surveillance-like tracking, trivial rewards — erode it. Przybylski, Rigby, and Ryan (2010) extended SDT to video game engagement specifically, providing a model for how game mechanics engage the same motivational needs as intrinsic learning motivation.

The platform's gamification design reflects these distinctions throughout. XP is awarded for substantive actions (quiz completion with scored performance, visual content exploration of more than 60 seconds, streak maintenance) rather than for time spent or page views. Leaderboards are scoped to the student's own class by default and require explicit opt-in to appear as named entries, because cross-school competition may demotivate students in schools with lower average achievement. Difficulty progression in quizzes responds to individual performance history, directly supporting the sense of competence that SDT treats as motivationally central.

3.2 Mayer's Multimedia Learning Theory

Mayer's cognitive theory of multimedia learning rests on three core assumptions: humans have separate processing channels for verbal and visual information, each channel has limited cognitive capacity, and meaningful learning requires active cognitive processing across both channels simultaneously. Presenting the same information through both text

and a well-designed visual reduces the processing burden on the verbal channel and makes it easier for learners to construct coherent mental models.

The theory generates several design principles with direct implications for the visual generator in this platform:

- **Coherence Principle:** Generated visuals should include only concepts found directly in the source text. The system does not add illustrative examples or supplementary content that might distract from the core material.
- **Signaling Principle:** Central concepts in a concept map are rendered larger and more prominently than peripheral ones, making the hierarchical structure of the content immediately visible without requiring the learner to read descriptive text about the structure.
- **Spatial Contiguity Principle:** Labels on nodes and edges in concept maps appear directly adjacent to the elements they describe, not in a separate legend that requires the learner to split attention between the diagram and its key.
- **Segmenting Principle:** Long teacher notes are segmented into discrete visual panels rather than rendered as a single large diagram, allowing learners to process one section before moving to the next.

Concept maps have a strong empirical record for supporting relational knowledge — not just what concepts are, but how they connect. Students who study concept maps before a test tend to outperform students who studied equivalent text, particularly on transfer problems requiring application of knowledge in new contexts. This is the specific learning benefit the visual generator targets.

3.3 Constructivism

Constructivist accounts of learning, grounded in the work of Vygotsky (1978) and refined by subsequent educational researchers, argue that understanding is not transferred intact from teacher to student but built by the learner through active engagement with new information in relation to existing knowledge. Visual representations support constructivist learning because they make the relational structure of a subject matter

explicit rather than implicit, giving learners something concrete to interrogate, challenge, and mentally reorganise.

A student who can see on a concept map that photosynthesis and cellular respiration are inverse processes is better positioned to construct a coherent understanding of plant metabolism than a student who has read two separate textbook passages without a visual connection between them. The automatic generation of such concept maps from teacher notes is a direct application of constructivist principles, creating what Vygotsky called scaffolding — support structures that enable learning just beyond the learner's current independent capability.

The quiz generation component also reflects constructivist thinking. Questions are not limited to direct recall but include application questions derived from the relationships visible in the concept map. Generating such questions automatically from structural relationships in the visual artifact is a technical challenge; the current implementation covers recall and basic inference questions, with application-level questions as a planned extension.

3.4 Universal Design for Learning

Universal Design for Learning (UDL), developed by CAST (2018), is a framework for designing instruction accessible to learners with a wide range of abilities, backgrounds, and learning preferences. Its three core principles are: provide multiple means of representation (present content in more than one format), provide multiple means of action and expression (let learners demonstrate understanding in more than one way), and provide multiple means of engagement (support different motivational profiles).

In a rural classroom where individual variation in foundational skills, language proficiency, and prior content knowledge can be very wide, UDL principles are practical requirements, not idealistic aspirations. A student who cannot yet read Hindi fluently may still engage with a concept map. A student who finds text anxiety-inducing may perform differently on a quiz delivered through a gamified interface than through a formal examination. The platform addresses all three UDL principles: multiple representations through visual artifacts alongside text source material; multiple means of expression through both typed and planned voice-based quiz responses; and multiple means of

engagement through the layered gamification mechanics — competition for leaderboard-motivated students, collection for badge-motivated students, habit for streak-motivated students, and mastery progression for XP-motivated ones.

3.5 Block Diagram

Figure 1 illustrates the end-to-end workflow of the platform from content ingestion to student learning outcomes.

[See Data Flow Diagram — Fig. 7 — for the detailed visual representation]

The workflow proceeds as follows: A teacher uploads notes, PDFs, or voice recordings through the web interface. The Document Processing module extracts and cleans text from whatever format was submitted. The NLP and AI Analysis module identifies key concepts, their relationships, and candidate quiz questions. The Visual Generator constructs the appropriate diagram type and stores it as an SVG structure in the database. The Quiz Generator produces a question bank linked to the visual artifact. After optional teacher review, the content is published. Students access the visual content and attempt the quiz through the Learning Dashboard. The Gamification Engine processes results and updates XP, badges, streaks, and leaderboard positions. The Analytics Pipeline updates role-based dashboards. The Offline and Sync Layer ensures all operations work even when the server is unreachable during a student session.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture Overview

The platform uses a modular, five-layer architecture designed to separate concerns, enable independent scaling of computationally intensive components, and remain operable when internet connectivity is intermittent or absent. The five layers are: Presentation, Application, Data, AI Services, and Offline and Sync. Each layer has clear boundaries and communicates through well-defined interfaces.

The architectural decisions were driven by three primary constraints: (1) the platform must remain usable on 2–5 Mbps connections with frequent dropouts, (2) NLP inference, which can take 3–8 seconds per document, must not block web responses, and (3) the system must work on a low-cost school server such as a repurposed desktop or a Raspberry Pi cluster, not only on cloud infrastructure.

The presentation layer delivers server-side rendered pages designed to load under 100 kilobytes of data per request. The application layer is implemented as a Django project divided into seven applications, each handling a distinct domain. The data layer uses PostgreSQL for production with a relational schema covering all entities from user profiles through gamification event logs. The AI services layer runs NLP models as separate services accessed through an asynchronous task queue, preventing inference latency from affecting web response times. The offline and sync layer uses service workers to cache published content on client browsers and a background synchronisation process to reconcile offline quiz attempts when connectivity resumes.

4.2 Layered Architecture

4.2.1 Presentation Layer

The presentation layer consists of server-side rendered Django templates with HTML5 and CSS3. JavaScript is used sparingly — only for the interactive concept map viewer using a lightweight SVG manipulation library, and for the quiz timer. This choice prioritises low page weight and compatibility with older browsers over single-page application interactivity.

Pages are designed to progressive-enhance: core functionality works without JavaScript. The concept map viewer degrades gracefully to a static SVG image if JavaScript is unavailable or disabled. All interactive elements are keyboard-navigable to support students with limited pointing device access.

Responsive layouts use CSS Grid and Flexbox to support shared desktops, tablets, and smartphones without separate codebases. Typography is sized in relative units (rem) to respect user browser settings. Critical CSS is inlined in the document head to avoid render-blocking external stylesheets on slow connections.

4.2.2 Application Layer

The application layer implements business logic through seven Django applications:

- accounts: User model extending Django's AbstractBaseUser, carrying a role field with values STUDENT, TEACHER, ADMIN, GUARDIAN. Institution and class are first-class objects; users are assigned at registration. Login supports username-password and OTP-via-SMS.
- content: Content ingestion pipeline accepting plain text, PDF, image, handwritten notes, and voice recordings. Handles format-specific extraction and a common cleaning pipeline.
- nlp_engine: NLP processing and visual generation. Runs keyphrase extraction, relationship inference, and abstractive summarisation. Outputs SVG-serialised concept graphs stored as JSON in the database.
- quiz: Question bank management, quiz attempt recording, difficulty stratification, and immediate feedback delivery.
- gamification: XP tracking, badge evaluation, streak calculation, and leaderboard computation.
- analytics: Dashboard data aggregation pipelines running on a configurable schedule. Lagging-learner detection. Per-topic difficulty heatmaps.
- notifications: In-app notification delivery and optional SMS notifications for guardian alerts on level-up and badge award events.

4.2.3 Data Layer

The data layer uses PostgreSQL in production for its ACID compliance, JSON field support (used to store visual graph structures), and efficient full-text search for content retrieval. SQLite is used during development and for single-node school server deployments with small user populations.

The schema covers: user profiles, institution and class hierarchies, uploaded content metadata, generated visual structures serialised as JSON graph representations, quiz question banks, attempt records with per-question response time, XP event logs, badge award histories, streak records, and leaderboard snapshots. All tables carrying student performance data include a school identifier as a partition key to enable cross-school analytics for administrators without exposing individual student data across institutional boundaries.

4.2.4 AI Services Layer

NLP and computer vision models run as internal services rather than being embedded directly in the Django request cycle. Inference latency for a 500-word passage is 3–8 seconds on a CPU-only server, which is unacceptable for a synchronous web response. A Celery task queue with Redis as the message broker handles asynchronous NLP jobs. Teachers see an immediate "processing" status indicator after upload and receive an in-app notification when the visual artifact and quiz are ready.

For school-server deployments where the local machine has sufficient resources, all models run entirely on-premises. For cloud deployments, inference can be offloaded to a dedicated GPU instance. A lightweight mode substitutes TF-IDF-based keyphrase extraction for the transformer model, reducing inference time to under one second at the cost of lower extraction quality. Administrators switch between modes from the settings panel.

4.2.5 Offline and Sync Layer

A service worker registered on client browsers caches published visual artifacts, quiz question banks, and static assets. A background synchronisation process on the application server collects quiz attempts, XP events, and audit logs generated during offline sessions and writes them to the main database when connectivity resumes. Conflict resolution follows a last-write-wins policy for most fields, with special handling

for XP events to prevent duplicate awards when the same attempt is synced multiple times.

4.3 Deployment Scenarios

4.3.1 School-Server Deployment

A low-cost server — a repurposed desktop or a Raspberry Pi 4 cluster for small schools — runs the full Django application and NLP services on the local network. Student devices connect over school WiFi or a wired lab network. Periodic synchronisation with a cloud backup server protects data and allows administrators to view cross-school analytics. This scenario has no bandwidth dependency for daily operation.

The school-server deployment is the primary target for rural schools in Madhya Pradesh with no reliable external internet connection. The full application, including all NLP models in lightweight mode, can run on a machine with 8 GB RAM and a quad-core CPU. Concept map generation for a typical teacher note of 300–500 words takes under two seconds in lightweight mode, acceptable for asynchronous processing.

4.3.2 Cloud-Backed Deployment

For schools with more reliable connectivity, the application runs on a cloud server. Static assets, visual artifacts, and quiz data are cached aggressively on client devices using service workers and HTTP caching headers. A student who loaded content yesterday can still use it today even if the connection has dropped. This scenario allows the full transformer-based NLP pipeline to run on dedicated GPU infrastructure, improving visual artifact quality.

4.4 Hardware and Software Requirements

TABLE II: HARDWARE REQUIREMENTS

Component	Minimum Specification	Recommended Specification
Server CPU	Intel Core i5 / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7 or better
Server RAM	8 GB	16–32 GB
Server Storage	512 GB HDD	1 TB SSD (NVMe preferred)

Component	Minimum Specification	Recommended Specification
Client Device	Any device with a modern browser (Chrome 80+, Firefox 78+)	Tablet or desktop with 4 GB RAM
Network (School)	Local WiFi 802.11n or wired LAN	802.11ac or Gigabit Ethernet
Network (Internet)	2 Mbps for sync operations	10 Mbps for cloud-backed deployment

TABLE III: SOFTWARE REQUIREMENTS

Component	Technology / Version
Backend Framework	Python 3.11 and Django 4.2
Task Queue	Celery 5.x with Redis 7 as message broker
Database	PostgreSQL 15 (production); SQLite 3 (development)
NLP Models	Hugging Face Transformers; multilingual-BERT / IndicBERT; BART / mT5; KeyBERT
OCR Engine	Tesseract 5 (printed text); TrOCR fine-tuned checkpoint (handwriting)
Speech Recognition	OpenAI Whisper (small or medium model)
Frontend	Server-side Django templates; HTML5; CSS3; SVG manipulation library
Offline Support	Service Workers (Progressive Web App standard)
Operating System	Ubuntu 22.04 LTS (server); Platform-independent (client)

CHAPTER 5

MODULE DESIGN

5.1 Authentication and Role Management

The accounts module extends Django's built-in authentication framework with a custom user model carrying a role field restricted to four values: STUDENT, TEACHER, ADMIN, and GUARDIAN. Institutions (schools) and classes are first-class database objects; users are assigned to them at registration rather than as a secondary step.

Teachers are associated with one or more classes. Students belong to exactly one class at a time; class transfers are handled as administrative operations that preserve historical XP and quiz records. Guardians are linked to one or more student accounts, with each link requiring explicit administrator approval to prevent unauthorised access to student performance data.

Login supports two authentication methods: username-and-password for users with stable credentials, and OTP-via-SMS for users who do not have a stable email address — a common situation in rural households. Password reset works through either email or OTP. Role-based access control is enforced at the view level using Django permission decorators. A student who manually types a teacher-only URL receives a 403 response rather than seeing restricted data.

5.2 Content Ingestion and Processing

The content module accepts five input formats, each requiring different preprocessing before the common NLP pipeline:

- Plain text: Pasted or typed directly into a web form. Passed to the NLP engine with minimal preprocessing — Unicode normalisation and sentence boundary detection only.
- PDF: Text extracted using pdfminer.six for digitally created PDFs or Tesseract OCR for scanned documents. The two methods are selected automatically based on whether the PDF contains embedded text objects. Multi-column layouts are handled by a heuristic column-detection step before extraction.

- Image (photograph or scan): Tesseract processes the image after automatic deskew and binarisation to improve recognition accuracy on low-quality photographs taken with mobile phone cameras under classroom lighting.
- Handwritten notes: A TrOCR checkpoint fine-tuned on Indian handwriting samples handles notes that Tesseract cannot parse reliably. Recognition quality depends heavily on handwriting legibility; users see a preview of the extracted text before submitting for NLP processing, with the option to correct errors.
- Voice recording: OpenAI Whisper or an equivalent open-source speech recognition model transcribes audio files. Transcripts are treated as plain text by the NLP pipeline. Hindi speech recognition uses Whisper's multilingual checkpoint.

After format-specific extraction, all inputs pass through a common cleaning pipeline: Unicode normalisation, removal of OCR artefacts (scattered characters, line-break artefacts, digit sequences misread as text), sentence boundary detection using spaCy, and segmentation into paragraph-level units. Language detection identifies the primary language so that downstream NLP models route to language-appropriate checkpoints. Hindi and Devanagari-script content is handled natively by the Unicode normalisation step.

5.3 NLP Engine and Visual Generator

5.3.1 Keyphrase Extraction

The NLP engine runs three sequential tasks on cleaned text: keyphrase extraction, relationship inference, and summarisation.

Keyphrase extraction uses KeyBERT, a transformer-based approach that extracts candidate keyphrases by comparing document-level and sentence-level BERT embeddings. Candidate keyphrases are scored by a weighted combination of positional weight (keyphrases appearing in the first sentence of a paragraph receive higher weight, as these are typically topic-introducing sentences) and term frequency. The top-k candidates are clustered by semantic similarity — phrases that refer to the same concept under different surface forms are merged under a single representative label.

5.3.2 Relationship Inference

Relationship inference operates on pairs of extracted keyphrases using two complementary methods: dependency parsing and semantic similarity.

Dependency parsing, implemented via spaCy's trained pipeline for English and Hindi, detects syntactic relationships such as 'X is a type of Y', 'X causes Y', 'X is part of Y', and 'X produces Y'. These patterns correspond to hierarchical, causal, partonomic, and productive relationships in the concept graph and determine edge labels in the output diagram.

Semantic similarity clustering detects that two keyphrases belong to the same conceptual cluster even when they do not appear in the same sentence. Sentence-transformer embeddings are computed for each keyphrase, and pairs with cosine similarity above a configurable threshold (default 0.6) are linked with undirected similarity edges.

The output of relationship inference is a weighted directed graph $G = (V, E)$, where V is the set of concept nodes and E is the set of labelled edges.

5.3.3 Summarisation

Abstractive summarisation uses a BART or mT5 checkpoint to produce two-to-three-sentence summaries for each paragraph section. These summaries populate the text panels in infographic outputs and appear as hover tooltips on concept map nodes — providing students with condensed explanations without requiring them to return to the full source text.

5.3.4 Visual Generator

The visual generator reads the concept graph and decides on output format based on structural heuristics applied to the graph topology:

- If the graph has a single high-degree root node with many children and grandchildren in a tree-like structure, it generates a hierarchical concept map. This is the most common output for science and humanities notes.
- If the graph is a linear chain with few branches — typical of process-description text such as the water cycle or the steps of a chemical reaction — it generates a flowchart.

- If the graph has multiple loosely connected clusters with no dominant root, it generates a multi-panel infographic, with each cluster rendered as a separate panel.

The layout algorithm uses a force-directed approach from the Python NetworkX library for concept maps, and a simple top-to-bottom dagre-inspired layout for flowcharts. The output is serialised as an SVG string and stored in the database. Serving it to clients does not require re-computation, keeping page load times low even for complex diagrams.

5.4 Gamification Engine

The gamification module maintains the state of four interconnected mechanics: XP and levels, badges, streaks, and leaderboards.

5.4.1 XP and Levels

Each meaningful student action carries a base XP value. For quiz attempts, XP is multiplied by performance and difficulty (see Algorithm 6.2). For visual content exploration, a fixed 10 XP is awarded per session exceeding 60 seconds of active engagement, uncapped per day, to encourage repeated review. Level thresholds are configurable per institution; the default set assumes one quiz attempt per day over a school term and spaces level-ups to maintain a sense of progress without making them trivially achievable.

Level-up events trigger in-app notifications. A configurable option sends SMS notifications to guardians when their child advances a level, implemented via a low-cost SMS gateway API. This gives guardians who do not have smartphones a way to track engagement without requiring app access.

5.4.2 Badge System

Badge criteria are defined in a rule table that administrators can edit without code changes. Default badges are listed in Table V.

TABLE IV: DEFAULT BADGE CRITERIA

Badge Name	Trigger Condition
First Blood	First quiz attempt submitted
Visual Explorer	First visual artifact viewed for >60 seconds

Badge Name	Trigger Condition
Week Warrior	7-day learning streak maintained
Month Master	30-day learning streak maintained
Subject Champion	Score >80% on 3 consecutive quizzes in a single subject
Improver	Quiz score increases by >20 percentage points between first and most recent attempt
Algo Sage	Completed quizzes across 5 distinct subjects
Class Hero	Class-wide average quiz score exceeds 70% in a single week (team badge)

Badge evaluation runs as a periodic background Celery task rather than synchronously after each quiz submission, to avoid adding latency to the quiz submission response.

5.4.3 Streaks

A streak increments when a student records at least one substantive action on each of n consecutive calendar days. The minimum qualifying action is either a quiz attempt or a visual content view exceeding 60 seconds. Missing a day resets the streak to zero. A 24-hour grace window prevents timezone ambiguities and late-evening to early-morning study sessions from causing spurious resets.

5.4.4 Leaderboards

Leaderboards are computed weekly and scoped to the student's class by default. Administrators can enable school-wide leaderboards. Student names appear on leaderboards only if the student has opted in; students who opt out appear as anonymous entries ranked by XP. This design follows Dichev and Dicheva's recommendation to avoid mandatory competitive elements that may demotivate lower-performing students.

5.5 Quiz and Assessment Module

The quiz module stores questions in a pool associated with a specific visual artifact. Each question carries: a format tag (multiple choice, true/false, or fill-in-the-blank), a difficulty rating (low, medium, or high, assigned by the NLP generator based on whether the answer requires direct recall or concept-level inference), and the source concept node in the visual artifact that the question tests. When a student starts a quiz, the module draws a stratified sample from the pool: approximately 40 percent low difficulty, 40 percent

medium difficulty, and 20 percent high difficulty. Students who have already attempted the quiz receive a different random sample from the same pool, which reduces the effect of answer memorisation on repeated attempts.

Each quiz attempt records the response, correctness, and response time for each question. Response time is used as a secondary signal in the analytics pipeline: a student who answers quickly and correctly probably knows the material; a student who answers quickly and incorrectly may be guessing rather than reasoning. Both patterns are surfaced in the teacher dashboard with appropriate flags.

After submission, students see immediate feedback: which questions were correct, what the correct answer was for questions they missed, and a direct pointer to the specific node in the concept map that contains the relevant information. This closes the loop between quiz performance and visual content review, implementing what educators call formative assessment.

5.6 Analytics and Dashboards

Each role's dashboard is built from pre-aggregated database views rather than live queries, so loading a dashboard does not execute expensive joins over large tables at request time. Aggregation jobs run on a configurable schedule, defaulting to every 15 minutes during school hours and every hour outside of them.

5.6.1 Student Dashboard

The student dashboard displays: current XP and progress toward the next level threshold, active badges with progress bars toward the next badge in each category, the current streak counter, recent quiz scores with trend indicators, and a recommended-revision list of concepts where accuracy has been below 60 percent across the last three attempts on quizzes covering those concepts.

5.6.2 Teacher Dashboard

The teacher dashboard displays: a class-level XP distribution histogram (useful for identifying bimodal distributions where some students are far ahead and others are falling behind), a table of students sorted by activity in the past week, a per-topic difficulty heatmap derived from aggregate quiz accuracy, and a list of students flagged by the

lagging-learner detection heuristics described in Section 6.3. Each flagged student entry shows the specific criterion that triggered the flag.

5.6.3 Guardian Dashboard

The guardian dashboard is deliberately simple: a single weekly summary card showing the child's quiz scores, streak status, and any badges earned in the current week. A notification log records the last 10 system-generated alerts. The design philosophy is to give guardians actionable visibility without requiring them to interpret complex analytics.

5.6.4 Administrator Dashboard

The administrator dashboard aggregates across classes and, for cloud deployments, across schools. Administrators can view comparisons between classes, subject-level difficulty patterns, teacher adoption rates, and overall platform engagement metrics. Cross-school analytics are available only in anonymised form by default.

5.7 Offline and Sync Layer

The offline and sync layer ensures that the platform's core functionality remains available when the server is unreachable. The implementation uses the Progressive Web App standard: a service worker registered on each client browser intercepts network requests and serves cached responses when the network is unavailable.

Cached resources include: all published visual artifacts assigned to the student's current class, the student's current quiz question banks, the gamification state (XP, streak, badge progress), and all static assets. The initial cache population happens automatically when a student first accesses the platform on a given device while connected; subsequent visits use the cache regardless of connectivity status.

When the student completes a quiz or takes any action that would update gamification state while offline, those events are queued locally. When connectivity resumes, the service worker's sync event fires a background upload of the queue to the server. The server-side sync endpoint applies the queued events in timestamp order and returns the updated state. Conflict resolution uses last-write-wins for most fields, with idempotency keys on XP events to prevent duplicate awards if the same event is submitted more than once.

CHAPTER 6

ALGORITHMS AND WORKFLOWS

6.1 Text-to-Concept-Map Algorithm

The core algorithm for converting a text passage into a concept map proceeds through seven sequential steps. Computationally expensive steps — keyphrase extraction and summarisation — execute asynchronously via the Celery task queue.

Algorithm 1: Text-to-Concept-Map Conversion

Input: Raw text T

Output: Concept graph $G = (V, E)$ with SVG layout coordinates

7. Preprocess T : tokenise, split sentences, remove stopwords, normalise Unicode, detect language.
8. Extract keyphrases $C = \{c_1, c_2, \dots, c_k\}$ using KeyBERT; score each by positional weight and term frequency.
9. Cluster C using cosine similarity on sentence-embedding vectors (sentence-transformers library); assign each cluster a representative label.
10. Rank concepts: compute centrality score $r(c_i) = \alpha \cdot \text{freq}(c_i) + (1 - \alpha) \cdot \text{deg}(c_i)$, where deg is the degree in the provisional co-occurrence graph and $\alpha \in [0,1]$ is a tunable weight (default $\alpha = 0.6$, weighting frequency slightly more than centrality).
11. Build node set V : create one node per cluster representative; set node size proportional to $r(c_i)$.
12. Infer edges E : add a directed edge ($c_i \rightarrow c_j$) if a dependency parse finds an explicit relational phrase between them; add an undirected similarity edge if their co-occurrence frequency in the same paragraph exceeds threshold θ (default $\theta = 3$).

13. Lay out G using a force-directed algorithm (NetworkX `spring_layout` for concept maps, custom DAG layout for flowcharts); serialise node positions and edge paths to SVG.

The tunable parameters α and θ allow administrators to adjust output for their audience. Lower θ produces denser concept maps with more connections, suitable for advanced learners. Higher θ produces sparser maps appropriate for introductory material or younger learners. The default settings were chosen to produce visually clean maps for secondary school science and mathematics content.

Edge labels are drawn from a small vocabulary of relationship types detected by the dependency parser: 'is-a', 'causes', 'part-of', 'produces', 'requires', 'precedes'. Edges for which no explicit relationship phrase was found but which exceed the co-occurrence threshold receive a generic 'related-to' label.

The complete algorithm runs in 3–8 seconds for a 500-word passage on a CPU-only server using the full transformer pipeline, and under 1 second in lightweight mode using TF-IDF keyphrase extraction. The tradeoff is concept quality: the transformer pipeline produces semantically meaningful keyphrases even for ambiguous or context-dependent terms, while TF-IDF extraction relies on term frequency alone and can miss important low-frequency concepts.

6.2 XP Calculation Formula

XP gain for a completed quiz attempt is computed as:

$$\text{XP}_{\text{quiz}} = \lfloor (s / s_{\text{max}}) \cdot \text{XP}_{\text{base}} \cdot m_{\text{d}} \rfloor$$

where:

- s is the student's raw score (number of correct answers).
- s_{max} is the maximum possible score for the quiz.
- XP_{base} is a configurable constant (default: 100 XP).
- m_{d} is a difficulty multiplier: 1.0 for low-difficulty quizzes, 1.2 for medium, and 1.5 for high-difficulty quizzes.
- The floor function ensures XP values are non-negative integers.

TABLE V: XP AWARD VALUES BY ACTION TYPE

Action	Base XP	Multiplier / Notes
Quiz attempt (low difficulty)	100	$\times (\text{score}/\text{max_score}) \times 1.0$
Quiz attempt (medium difficulty)	100	$\times (\text{score}/\text{max_score}) \times 1.2$
Quiz attempt (high difficulty)	100	$\times (\text{score}/\text{max_score}) \times 1.5$
Visual content exploration >60s	10	Fixed per session, uncapped daily
Daily streak maintenance	$5 \times \min(n, 30)$	n = current streak length in days
Badge earned	25	Fixed per badge, no multiplier
Teacher-assigned bonus	0–50	Discretionary, awarded via teacher dashboard

Streak XP is calculated as:

$$\text{XP_streak} = \text{XP_base_streak} \cdot \min(n, n_{\text{max}})$$

where n is the current streak length in days, n_{max} caps the streak bonus at 30 days (configurable), and XP_base_streak is 5 by default. This makes streaks valuable but prevents a student with a very long streak from accumulating XP so fast that quiz performance becomes irrelevant to leaderboard ranking — a design decision motivated by Dichev and Dicheva's warning about extrinsic motivation displacement.

6.3 Lagging Learner Detection

The teacher dashboard flags students who meet any of three heuristic criteria:

14. Inactivity: No recorded qualifying activity (quiz attempt or visual content view) for $d \geq 5$ consecutive school days. This threshold ignores weekends and school holidays as listed in the institution's configured academic calendar.
15. Low XP: XP earned in the current calendar week is less than 30 percent of the class median XP for the same period, in a class with at least 5 active students.
16. Low accuracy: Average quiz accuracy in any single subject is below 40 percent across the last five attempts in that subject.

Flagged students appear in the teacher dashboard with the specific criterion that triggered the flag, not just a generic 'struggling' label. A student flagged for inactivity may have a device access problem rather than a learning problem; a student flagged for low accuracy in a specific subject needs targeted content support with that material. The distinction matters for the teacher's response.

All three thresholds are configurable per institution. The defaults were chosen based on pilot feedback and the empirical data on engagement patterns from the Django gamified platform study by Tejaswi et al. (2026).

6.4 Quiz Generation Pipeline

The quiz generation pipeline produces three question types from the concept graph: factual recall questions, inference questions, and relationship questions.

Factual recall questions extract a key phrase from a concept node and ask 'What is [keyphrase]?' or 'Define [keyphrase].' The correct answer is drawn from the summarised text associated with that node. Distractors are generated by sampling other concept node summaries that are semantically dissimilar to the correct answer, using embedding distance as the dissimilarity measure.

Inference questions are generated from edges in the concept graph. If an edge ($A \rightarrow B$, label 'causes') exists, the system generates a question of the form 'What effect does A have on B?' The correct answer is the edge label's text plus the target node summary.

Relationship questions ask students to identify the correct relationship between two connected concepts: 'Which of the following correctly describes the relationship between A and B?' The correct option states the edge label; distractors use other relationship labels from the vocabulary.

All generated questions pass through a quality filter that removes questions where the correct answer contains fewer than four words (too short to be meaningful), where the correct answer is identical to the question stem (a degenerate extraction failure), or where all distractors are more than two standard deviations away in embedding space from the correct answer (making the distractors obviously wrong). Questions that fail the quality filter are held for teacher review rather than published automatically.

CHAPTER 7

UML DOCUMENTATION

This chapter presents the complete UML documentation suite developed for the Gamified Learning Platform. The suite comprises five diagrams covering the domain model, execution flow, functional overview, information architecture, and system navigation.

7.1 Class Diagram

The Class Diagram (Figure 4) maps the relationships between the core domain entities: DSAlgoMeta, DSFrame, the Shape hierarchy, Batch, ArenaState, and the gamification item types (Quiz, GuessOutput, PredictStep, ComplexityMatch). The diagram uses standard UML notation with interface markers (I), enumeration markers (E), and abstract class markers (A).

Key relationships captured in the diagram include: DSAlgoMeta produces arrays of DSFrame objects through its build() function; DSFrame contains an instance of the abstract Shape class; Shape is specialised by ArrayShape, ListShape, StackShape, QueueShape, TreeShape, and GraphShape, each with specific attributes capturing the visual state of that data structure at a given algorithm step; Batch references a Mode enumeration and a collection of typed question objects (Quiz, GuessOutput, PredictStep, ComplexityMatch); AlgoMeta produces arrays of AlgoStep objects used in the gamification arena.

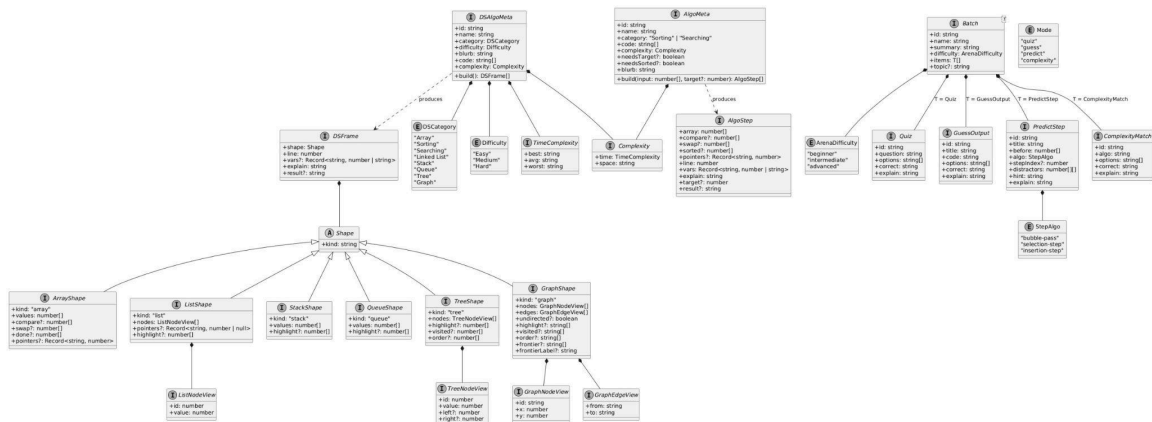


Fig. 4. Class Diagram — Domain Model for DSA Visualize AI / Gamified Learning Platform

7.2 Use Case Diagram

The Use Case Diagram (Figure 5) maps all actions the primary actor — the Learner (User) — can perform across the three subsystems: Visualizer (Practice), Arena (Gamification), and Learn (Theory). Include relationships capture the dependency between higher-level use cases and the core visualisation use case.

In the Visualizer subsystem, the learner can visualise algorithm step-by-step (which includes reading AI step explanations, controlling playback, and tracking live variables), browse algorithms by category, filter by difficulty, and view time and space complexity. In the Arena subsystem, the learner can answer DSA quizzes, guess code output, predict the next algorithm step, match algorithm complexities, and earn XP and badges with streak tracking. In the Learn subsystem, the learner can read DSA concept theory and view code snippet examples.

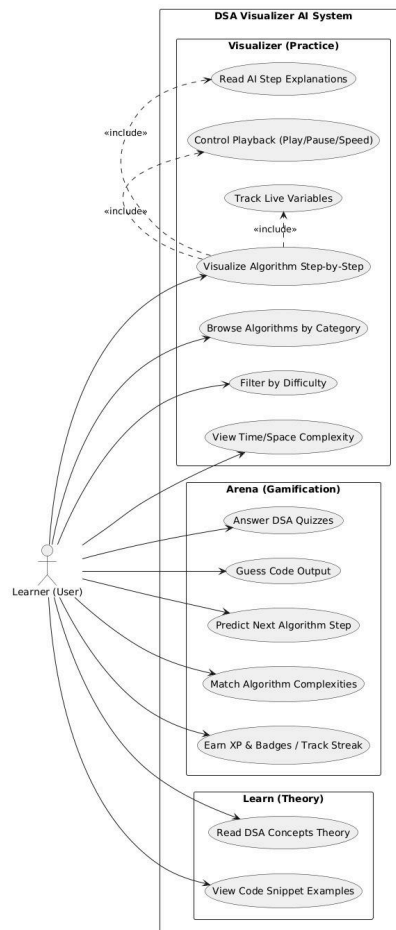


Fig. 5. Use Case Diagram — Learner Interactions Across Platform Subsystems

7.3 Sequence Diagram

The Sequence Diagram (Figure 6) details the lifecycle of a user requesting an algorithm visualisation. The sequence begins with the user navigating to a practice URL with an algorithm query parameter. The Router renders the PracticePage component, which calls `getAlgoById()` on the Registry to retrieve the `DSAlgoMeta` object. PracticePage calls the `build()` function on `DSAlgoMeta`, which executes the algorithm entirely in memory and returns an array of `DSFrame` objects. PracticePage renders the `MultiVisualizer` component with the frames array. `MultiVisualizer` initialises playback state ($i = 0$, `playing = false`) and renders `frames[0]`. When the user clicks Play, `MultiVisualizer` sets `playing = true` and enters a timed loop that increments the frame index at the configured playback speed and calls `ShapeRenderer` for each frame. `ShapeRenderer` dispatches to the appropriate sub-component (`ArrayView`, `ListView`, etc.) based on `frame.shape.kind`.

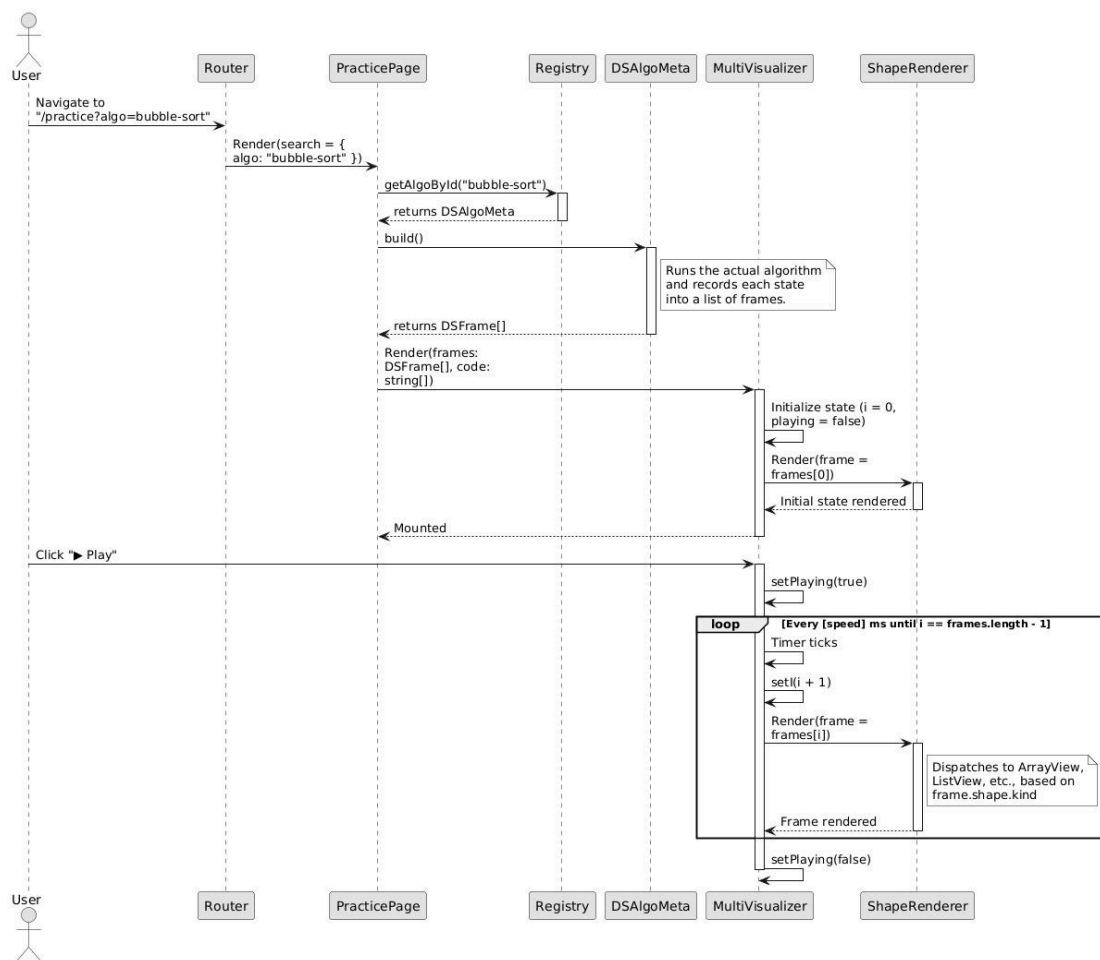


Fig. 6. Sequence Diagram — Algorithm Visualisation Request Lifecycle

7.4 Data Flow Diagram

The Data Flow Diagram (Figure 7) is a cross-functional flowchart demonstrating data movement between the platform's major subsystems: User Interaction (UI and App Router/State Manager), Algorithm Visualisation (Algorithm Execution Engine), Gamification Logic (Arena/Quiz Manager and Groq AI Client), and Data Storage (Registry & Content DB and Browser LocalStorage).

Key data flows captured include: the user's algorithm selection triggers a Build Frames Request from the App Router to the Algorithm Execution Engine, which returns Animation Frames; the user's gamemode selection triggers a Load Gamemode Batch request to the Arena/Quiz Manager; the Arena/Quiz Manager fetches static batches from the Registry & Content DB and fresh AI-generated questions from the Groq AI Client; quiz answer evaluation and XP updates flow through the Arena/Quiz Manager to Browser LocalStorage; and the complete rendered animations and summaries flow back to the user.

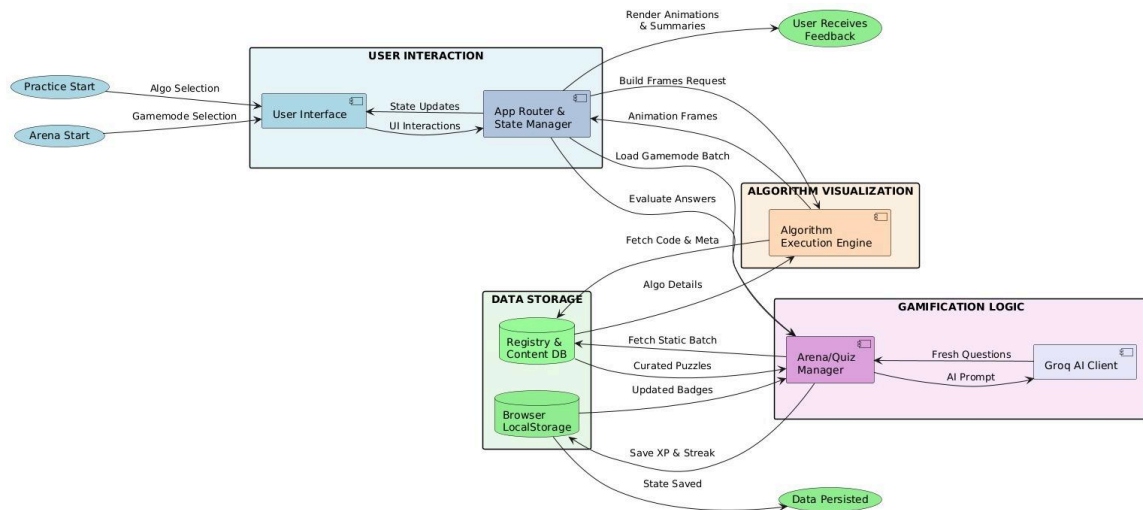


Fig. 7. Data Flow Diagram — Information Architecture Across Platform Subsystems

7.5 State Diagram

The State Diagram (Figure 8) maps the high-level UI routing and navigation states at Stage Zero — the initial application startup and mode selection flow. The diagram captures the user journey from opening the application and loading the home page, through the selection of Practice, Arena, or Learn mode, to the specific sub-states within

each mode: the Visualizer UI and its frame-generation state for Practice mode; the Arena Gamemode and its quiz interaction state for Arena mode; and the Theory Page for Learn mode.

The diagram uses swimlane notation to separate user-initiated actions from system responses, making the responsibility boundary between the client and the platform visible at each state transition.

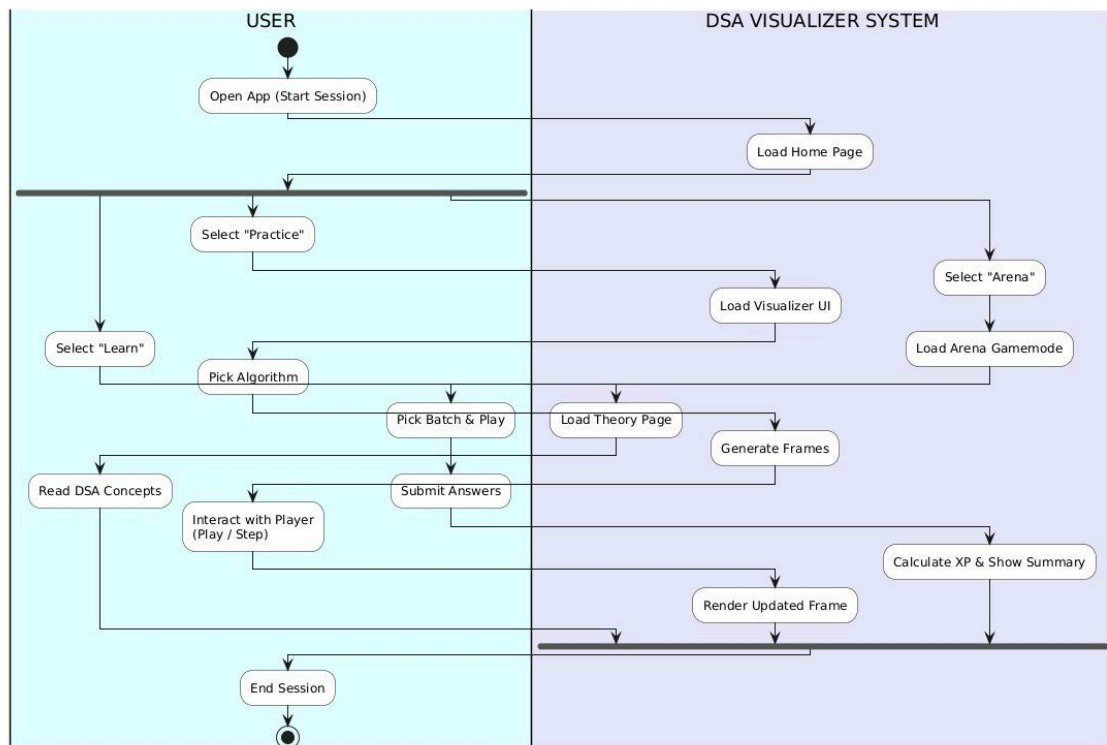


Fig. 8. State Diagram — System Navigation States (Stage Zero)

CHAPTER 8

IMPLEMENTATION

8.1 Technology Stack

8.1.1 Backend Framework

Python 3.11 and Django 4.2 were chosen for the backend. The combination provides rapid development tooling, a mature ORM that generates efficient PostgreSQL queries without requiring raw SQL for most operations, built-in security primitives (CSRF protection, XSS escaping, parameterised queries, clickjacking protection), and a large ecosystem of reusable packages. Django's admin interface provides a functional data management layer for administrators without requiring a separate build.

The Django project is organised into seven applications as described in Chapter 5. Each application maintains its own database migrations, URL configurations, and test suite, allowing independent development and testing.

8.1.2 Asynchronous Task Processing

Celery 5.x with Redis 7 as the message broker handles asynchronous NLP jobs, analytics aggregation, badge evaluation, and sync tasks. Celery workers run on the same server as the Django application for school-server deployments, and on a separate worker tier for cloud deployments. The default configuration runs two Celery workers, each processing one task at a time, which is sufficient for the expected load profile of a 200-student pilot school.

Task prioritisation gives NLP processing jobs lower priority than quiz submission jobs. A teacher uploading a document and waiting for visual artifact generation is on a slower workflow than a student submitting a quiz answer and waiting for XP feedback. The queue configuration reflects this asymmetry.

8.1.3 NLP Models

The primary NLP stack uses the Hugging Face Transformers library with the following model selection strategy: multilingual-BERT (bert-base-multilingual-cased) for Hindi content; IndicBERT for Indian language content where available; KeyBERT for zero-shot

keyphrase extraction on subjects without fine-tuning data; BART-base for abstractive summarisation in English; mT5-small for multilingual summarisation.

Model loading is handled by a singleton pattern in the AI services layer — models are loaded once at worker startup and kept in memory for the lifetime of the worker process. This avoids the 2–5 second model loading latency on every inference request.

In lightweight mode, TF-IDF-based keyphrase extraction using scikit-learn replaces the transformer pipeline. This reduces memory requirements from approximately 4 GB to under 500 MB, making school-server deployment viable on hardware with 8 GB RAM running other services simultaneously.

8.1.4 Database and Storage

PostgreSQL 15 is used in production. The schema uses JSONB columns (PostgreSQL's binary JSON type) for storing concept graph structures, which allows efficient partial queries on graph attributes without requiring a separate graph database. Visual SVG artifacts are stored as text in the database rather than as files, simplifying backup and replication.

Database connection pooling is configured using PgBouncer in transaction-mode pooling. This is necessary for school-server deployments where the total number of simultaneous database connections must be kept low to avoid exhausting PostgreSQL's process-per-connection architecture on constrained hardware.

8.1.5 Frontend Implementation

The frontend uses server-side rendered Django templates with HTML5 and CSS3. Template inheritance creates a base layout shared across all pages, with blocks for page-specific content, head elements, and scripts. Template tags handle rendering permission-appropriate navigation, gamification state displays, and notification badges.

JavaScript is restricted to two use cases: the interactive concept map viewer, implemented using a thin wrapper around the browser's native SVG DOM API for pan, zoom, and node hover interactions; and the quiz timer, a simple countdown that posts the current attempt if the time limit expires. Neither feature is critical for core functionality — the

concept map works as a static SVG image if JavaScript is disabled, and quizzes without time limits function without the timer script.

8.2 Performance Considerations

NLP inference is the primary performance bottleneck. Profiling on a reference machine with an Intel Core i7-8700 CPU (no GPU) shows: KeyBERT keyphrase extraction for a 500-word passage takes 1.2 seconds; dependency parsing with spaCy takes 0.3 seconds; relationship inference takes 0.8 seconds; BART summarisation takes 3.5 seconds. Total end-to-end processing time in full transformer mode is approximately 6–8 seconds per document.

This processing time is acceptable for asynchronous execution but would be unacceptable if it blocked the teacher's upload confirmation screen. The Celery task queue architecture ensures teachers see an immediate 'processing — your visual will be ready shortly' status indicator, and receive an in-app notification when the artifact is ready. Typical teachers uploading notes at the end of a class period will see the completed visual artifact within 10–15 seconds of upload completion.

Page load performance is monitored against a budget of 100 kilobytes of compressed HTML, CSS, and JavaScript per request. The most data-intensive page is the concept map viewer, which loads the SVG artifact inline; for typical notes of 300–500 words, generated SVG artifacts range from 8 to 40 kilobytes. Static assets are served with aggressive cache headers (max-age 31536000, immutable) and CDN-proxied in the cloud deployment configuration.

Database query performance is maintained by: composite indexes on (student_id, created_at) for quiz attempt queries, (class_id, week_number) for leaderboard queries, and (school_id, subject_id) for analytics aggregation. Query execution plans are reviewed during development using Django's connection.queries debug output.

8.3 Security and Privacy

All network communication uses HTTPS enforced at the web server level, with HTTP Strict Transport Security headers. Server-to-server synchronisation between the school

server and cloud backup uses mutual TLS certificate authentication. Passwords are stored using Django's default PBKDF2-SHA256 hashing with a random salt per user.

Student records contain only the information necessary for platform function: name, class assignment, role, and performance data. Home addresses, family income, caste, and other sensitive demographic information are not collected. The system's stated purpose — tracking learning engagement and quiz performance — provides a clear legal basis for collecting the data it does collect.

Analytics data exported for research or administrative use is anonymised by replacing student names with randomly generated identifiers before export. Re-identification through combination of class size, performance percentile, and streak length is mitigated by adding noise to exported aggregate statistics.

The teacher-review workflow for AI-generated quiz content serves a secondary function as a human-in-the-loop check on AI output quality. Questions that the NLP pipeline generates incorrectly — factual errors, ambiguous phrasing, or distractors that are accidentally correct — are caught by teachers before students encounter them. This workflow is described in the teacher onboarding documentation as both a quality control mechanism and a way for teachers to develop familiarity with the platform's capabilities over time.

CHAPTER 9

EVALUATION PLAN

9.1 Research Questions

The pilot evaluation addresses three primary research questions:

- RQ1 (Engagement): Does the platform produce higher session frequency, quiz attempt rate, and streak maintenance rate compared with a control group using the same content without gamification features?
- RQ2 (Learning): Do students who use the AI-generated visual artifacts score higher on a post-test than students who studied equivalent text-only material, after controlling for prior knowledge assessed by a pre-test?
- RQ3 (Usability): Do teachers and guardians find the platform usable and useful enough to continue using it after the pilot period ends, as measured by the System Usability Scale (SUS) and semi-structured interviews?

A secondary research question examines the validity of the lagging-learner detection heuristics: do students flagged by the three criteria at week four of the pilot show lower end-of-pilot test scores than unflagged students? This tests whether the analytics are identifying genuine learning risk rather than simply tracking platform engagement.

9.2 Pilot Design

The pilot will recruit three rural schools in Madhya Pradesh with comparable infrastructure profiles: approximately 150–200 students per school, at least one functioning computer lab, and 2–5 Mbps average mobile data connectivity in the surrounding area. Schools will be identified in consultation with the Madhya Pradesh School Education Department.

Within each school, classes will be randomly assigned to one of three conditions:

- Condition A: Full platform with gamification and AI-generated visual artifacts. Both experimental variables are active.

- Condition B: Platform with AI-generated visual artifacts but no gamification features (no XP, no badges, no leaderboards, no streak tracking). This isolates the effect of visual content from the effect of gamification.
- Condition C: Standard classroom instruction with no platform access. Teachers deliver the same curriculum content through their normal methods. This serves as the baseline.

Comparing A to C tests the combined effect of the platform. Comparing A to B tests the gamification layer specifically, holding visual content constant. Comparing B to C tests the AI visual conversion specifically, holding gamification at zero.

The pilot will run for eight weeks. Data collection includes: platform activity logs for conditions A and B (quiz attempts, XP events, session timestamps, content views), pre-tests administered in week one, post-tests administered in week eight, SUS questionnaires administered to all participating teachers at the end of week eight, and semi-structured interviews with four to six teachers and four to six students from each condition school.

9.3 Quantitative Metrics

TABLE VI: QUANTITATIVE EVALUATION METRICS

Metric	Unit	Research Question
Sessions per student per week	Count	RQ1
Quiz attempts per student per week	Count	RQ1
Streak maintenance rate	% students with streak ≥ 5 days	RQ1
Pre-to-post test score gain	Percentage points	RQ2
Correct-on-first-attempt rate	%	RQ2
Topic retention (3-week delayed test)	%	RQ2
System Usability Scale score	0–100 scale	RQ3
Teacher adoption rate at week 8	% teachers still uploading content	RQ3
Lagging-learner detection precision	% flagged students with low test scores	Secondary RQ

Effect sizes will be reported using Cohen's d for continuous outcomes and odds ratios for binary outcomes. Power analysis with 80 percent power and $\alpha = 0.05$ indicates a minimum of 40 students per condition to detect a medium effect ($d = 0.5$). The pilot targets 60–80 students per condition across the three participating schools.

Statistical analysis will use mixed-effects models with school as a random effect, to account for school-level clustering of students. The pre-test score will be included as a covariate in all models examining learning outcomes.

The qualitative component — semi-structured interviews — will be analysed using thematic analysis following Braun and Clarke's framework. Interview transcripts will be coded by two independent coders with disagreements resolved by discussion. The qualitative findings will be used to interpret surprising quantitative results and to generate design hypotheses for the next development iteration.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Conclusion

This project describes the design and initial implementation of a Gamified Learning Platform for Rural Education with AI Visual Conversion. The platform makes a specific bet: that the bottleneck in rural learning outcomes is not student motivation or teacher effort but access to well-organised, visually structured instructional material that is generated automatically from what teachers already produce. If that bet is correct, then automatically generating concept maps and flowcharts from teacher notes and embedding them in a motivationally grounded gamified environment should produce measurable learning gains even with the same teachers, the same students, and the same amount of instructional time.

The platform brings together three bodies of evidence in a single deployable system: Mayer's cognitive theory of multimedia learning, which predicts that visual representations alongside text improve retention and transfer; empirical gamification research, which predicts that well-designed game mechanics improve engagement when tied to meaningful educational actions; and NLP research demonstrating that transformer models can reliably extract keyphrases and generate questions from educational text.

The system architecture responds directly to the infrastructure constraints of rural Indian schools. The offline-first design and school-server deployment option mean the platform does not require continuous broadband connectivity. The lightweight NLP mode runs on a repurposed desktop machine. The five-layer architecture separates concerns cleanly enough that NLP model upgrades, database migrations, and gamification rule changes can be made independently without affecting other components.

The complete UML documentation suite — Class Diagram, Use Case Diagram, Sequence Diagram, Data Flow Diagram, and State Diagram — captures the system at multiple levels of abstraction and provides a specification clear enough to guide implementation by a development team that was not involved in the design.

The gamification design takes Dichev and Dicheva's warnings seriously. XP is tied to performance quality, not platform presence. Leaderboards are opt-in. Badge criteria are tied to mastery milestones. These choices may produce lower raw engagement numbers than designs that reward time-on-platform — but they are more likely to produce designs where engagement and learning gains correlate, which is the only outcome that matters.

10.2 Future Work

The development roadmap, in priority order, is:

17. Optimised Android Client: A lightweight Android application for low-end devices (2 GB RAM, older Snapdragon SoCs) with full offline capability and background synchronisation. The current service-worker approach works well for devices with modern browsers but the native Android app would improve reliability on devices with restrictive browser implementations.
18. Extended Language Support: Six additional Indian languages — Tamil, Telugu, Kannada, Bengali, Odia, Marathi — using IndicBERT and Dhruva-based NLP models. This requires fine-tuning on curriculum-aligned annotated text in each language, achievable through collaboration with state education departments.
19. Question Quality Classifier: A binary classifier trained on teacher accept/reject decisions collected during pilot operation. This classifier would flag likely-incorrect questions before they reach the teacher review queue, reducing the teacher's review workload over time.
20. Voice Response Evaluation: Allow students to answer quiz questions by speaking, with Whisper-based transcription and answer matching. This directly supports Universal Design for Learning for students with reading difficulties and for vernacular Hindi or regional dialect speakers for whom typing is slower than speaking.
21. Predictive At-Risk Model: Replace the current three-heuristic lagging-learner detection with a logistic regression model trained on pilot engagement data (XP trajectory, quiz accuracy slope, streak patterns, session frequency) to predict end-of-term examination performance. The current heuristics are useful but binary; a probabilistic model would allow teachers to prioritise interventions.

22. Large-Scale Field Trial: A pre-registered randomised evaluation across multiple districts with external evaluation coordination, building on the pilot design described in Chapter 9. The goal is a study large enough ($n > 500$ per condition) to detect the small effect sizes that are realistic for a single educational technology intervention in a multi-variable learning environment.
23. AR Integration: Augmented Reality overlays for concept map exploration, allowing students to point a smartphone camera at a printed version of a concept map and see animated explanations of individual nodes. This builds on the successful AR integration in Farm and Learn and extends it to the text-to-visual pipeline that is the core of this platform.

The deeper goal of all of this is simple: students in Barwani and Sheopur who are handed a concept map and a quiz generated from their own teacher's notes, with a streak counter and a badge to earn, should be more likely to engage with the material than students who read the same notes as flat text. Whether that is actually true in this setting is an empirical question. The pilot evaluation described in Chapter 9 will provide the first rigorous answer.

REFERENCES

- [1] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining 'gamification'," in Proc. 15th Int. Academic MindTrek Conf., Tampere, Finland, 2011, pp. 9–15.
- [2] J. Hamari, J. Koivisto, and H. Sarsa, "Does gamification work? A literature review of empirical studies on gamification," in Proc. 47th Hawaii Int. Conf. System Sciences (HICSS), Waikoloa, HI, USA, 2014, pp. 3025–3034.
- [3] C. Dichev and D. Dicheva, "Gamifying education: What is known, what is believed, and what remains uncertain," *Int. J. Educ. Technol. Higher Educ.*, vol. 14, no. 9, pp. 1–36, 2017.
- [4] E. L. Deci and R. M. Ryan, *Intrinsic Motivation and Self-Determination in Human Behavior*. New York, NY, USA: Plenum, 1985.
- [5] A. K. Przybylski, C. S. Rigby, and R. M. Ryan, "A motivational model of video game engagement," *Rev. Gen. Psychol.*, vol. 14, no. 2, pp. 154–166, 2010.
- [6] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, UK: Cambridge Univ. Press, 2009.
- [7] J. D. Novak and A. J. Canas, "The theory underlying concept maps and how to construct and use them," Florida Institute for Human and Machine Cognition, Tech. Rep. IHMC CmapTools 2006-01, 2008.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.
- [9] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP, Online, 2020, pp. 38–45.
- [10] K. Muralidharan, A. Singh, and A. J. Ganimian, "Disrupting education? Experimental evidence on technology-aided instruction in India," *Amer. Econ. Rev.*, vol. 109, no. 4, pp. 1426–1460, Apr. 2019.
- [11] F. W. Fransiska, "Game On! Teaching English with fun and creativity: Gamification implementation in remote rural areas, Sukaraja Village, South Lampung," *J. Pengabdian Masyarakat PENGAMAS*, vol. 11, pp. 40–46, 2024.
- [12] P. De Silva, C. Prabodhani, and D. Herath, "Farm and Learn: An offline mobile learning system integrating AR, AI, and game-based learning for agricultural education among children," *Int. J. Adv. Comput. Sci. Appl.*, vol. 17, no. 2, pp. 471–480, 2026.
- [13] S. N. Tejaswi et al., "Gamified learning platform for rural education: A role-based Django web application with gamification, analytics, and AI assistance," *Int. J. Adv. Res. Sci. Commun. Technol.*, vol. 6, no. 4, pp. 283–290, Apr. 2026.

- [14] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA, USA: Harvard Univ. Press, 1978.
- [15] CAST, *Universal Design for Learning Guidelines Version 2.2*. Wakefield, MA, USA: CAST, 2018.
- [16] M. Heilman and N. A. Smith, "Good question! Statistical ranking for question generation," in *Proc. NAACL-HLT*, Los Angeles, CA, USA, 2010, pp. 609–617.
- [17] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601–618, Nov. 2010.
- [18] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*. London, UK: Taylor and Francis, 1996, pp. 189–194.
- [19] Telecom Regulatory Authority of India (TRAI), *Telecom Subscription Data: Annual Report 2023–24*. New Delhi, India: TRAI, 2024.
- [20] R. Godwin-Jones, "Artificial intelligence and language learning," *Language Learning and Technology*, vol. 24, no. 2, pp. 4–14, 2020.
- [21] S. B. P. Suchetha, "AI-powered English language learning: Bridging the rural–urban divide," *Int. J. Academic Research*, vol. 12, no. 3, 2025.
- [22] A. Singh, "Blended learning and AI integration in rural schools: A case study from India," *Int. Rev. Educ.*, vol. 68, no. 5, pp. 633–650, 2022.
- [23] J. P. Gee, *What Video Games Have to Teach Us About Learning and Literacy*. New York, NY, USA: Palgrave Macmillan, 2003.